JuriSense Gate

Demo Presenter Script — read straight through, part by part

ACTION: Cold open on the JuriSense Gate login/dashboard, Acme Tecnologia Ltda. already signed in as the admin user.

This is JuriSense Gate — it turns contracts, supplier data, and company policies into auditable compliance decisions. Acme Tecnologia, a 200-person Brazilian SaaS company, needs to hire a new vendor. Before anyone signs anything, JuriSense Gate has to answer: can we hire this supplier, and sign this contract? This organization and its compliance policy were configured ahead of time — what you're about to see is the live supplier and contract workflow.

Part 1 — Supplier verification

ACTION: Create a new Supplier record, name "CloudDocs Software Ltda.", enter its CNPJ.

First, the supplier. I create CloudDocs Software as a new supplier — it starts in prospect status.

ACTION: Submit; show the SupplierCheck running, then its result.

Submitting the CNPJ triggers a SupplierCheck — a point-in-time verification against official sources: company registry status, and the CEIS, CNEP, and CEPIM sanctions lists. This check is immutable once it completes — we never overwrite a verification run, we create a new one. Registry status: active. No sanctions hits. The supplier moves to under_review.

Part 2 — Contract ingestion and AI extraction

ACTION: Upload clouddocs-saas-v1-original.pdf as a new Document / Contract.

Now the contract. I upload CloudDocs' proposed SaaS agreement. On upload, the system computes a SHA-256 hash of the document — this is what eventually chains into the Evidence Package.

ACTION: Show the AI extraction result with clause citations.

AI extraction runs against the contract and pulls out the terms that matter: contract value, R$480,000 a year. Term, 24 months. An automatic renewal clause. Unlimited liability. It processes personal data — yes. A Data Processing Agreement — absent. Every one of these fields traces back to a specific clause and page in the source document. This is provenance, not a black-box summary.

Part 3 — Policy Engine evaluation and the first decision

ACTION: Trigger a Compliance Review for this contract version.

Now I trigger a compliance review against Acme's published policy. Three rules fire. LGPD-001: the contract processes personal data but has no DPA — that's a BLOCK, critical severity, and it's not overridable without an explicit override action. LEGAL-008: unlimited liability — REVIEW, routed to Legal. FIN-003: contract value over R$250,000 — REVIEW, also routed to Legal.

ACTION: Show the aggregated Decision: BLOCK.

Here's the product's core explainability claim: even though two rules only said REVIEW, one rule said BLOCK — and BLOCK always wins, because it's the safer failure mode. The aggregation is a pure function of the set of outcomes, not the order they ran in. And the decision is never a bare status: why — missing DPA on a contract that processes personal data; evidence — the clause citation and extracted field; policy — LGPD-001; required action — add a DPA before this can proceed.

Part 4 — Correction and re-evaluation

CloudDocs corrects the contract: they add a DPA and cap liability at twelve months of fees instead of leaving it unlimited.

ACTION: Upload clouddocs-saas-v2-corrigido.pdf as a new Contract Version on the same Contract.

This is a brand-new contract version — never an edit to the old one, which stays exactly as it was. AI extraction runs again: DPA present, liability capped.

ACTION: Trigger a new Compliance Review; show the new aggregated Decision: REVIEW.

A new review runs. LGPD-001 now passes. Liability is no longer unlimited. But FIN-003 still fires — the contract value hasn't changed, it's still above the threshold. Aggregation now yields REVIEW, not APPROVE. This is realistic, not a scripted 'everything turns green' moment: a R$480k, 24-month contract still needs a human legal sign-off on value alone, even after the compliance blocker is fixed. The system doesn't over-claim automation where a human judgment call is genuinely warranted. And this new decision supersedes the original BLOCK — that old decision is never edited or deleted.

Part 5 — Human approval

ACTION: Switch to the legal-approver account; open the pending Approval Request.

Because this is REVIEW, an approval request routes to whoever holds the Legal role in this organization — not a hardcoded person. Logged in as Acme's Legal approver, I see the full rationale: which rules fired, which passed, the extracted clause citations. I approve it, with a justification note.

Part 6 — Decision Passport, evidence, and blockchain anchoring

ACTION: Open the Decision Passport view for this Decision.

With the decision resolved, the system packages a Decision Passport: the decision, the supplier, the contract, document hashes, the exact policy version evaluated, every rule evaluation — including the historical BLOCK — the external sources consulted, the AI's own execution provenance, the approver, and the

timestamp. Ask 'why was this approved?' two years from now, and this is the complete, reproducible answer — not 'someone probably looked at it.'

[Note for the presenter: the anchor for this Decision was pre-submitted a few minutes before this recording — Constellation's network needs time to reach consensus, so we don't sit here waiting on camera. Only the next step, Verify, is live.]

ACTION: Show the BlockchainAnchor record: anchor_ref, provider, submission timestamp.

An Evidence Package was built from this decision's evidence — the document hashes, the supplier check result, every rule evaluation — and a Merkle root computed over all of it. That root was submitted to Constellation Network. Nothing about this contract or this supplier's personal data goes on-chain — only a hash of the evidence bundle. The chain proves the evidence wasn't altered after the fact; it is not a copy of the evidence.

Part 7 — Public verification

ACTION: Open the public verification page for this anchor — no login.

Last step: the public verification page. No login required — that's the whole point of a public, third-party-checkable proof.

ACTION: Click "Verify" live; show the confirmation.

I click Verify, live, right now — and it confirms the hash matches what was anchored. Anyone — an auditor, a regulator, an acquirer's due diligence team, the vendor itself — can independently confirm this decision's evidence hasn't been tampered with, without trusting Acme's internal database, or JuriSense's word for it. That's the product's core promise, proven live.

ACTION: Hold on the confirmed verification result for the close.

That's JuriSense Gate: contracts, supplier data, and company policies — turned into auditable compliance decisions.

Production notes

A few things to keep in mind

If a judge asks what happens when the AI extraction is wrong: the AI only interprets and extracts, it never decides a compliance outcome by itself. The deterministic Policy Engine rules are what actually gate APPROVE, REVIEW, and BLOCK, and every extracted field carries a citation back to the source document so a human can check it.

Never imply the blockchain anchor was submitted live if it was pre-staged ahead of time — say so plainly if it comes up. Only the verification step needs to happen live.

Seed data for this walkthrough lives in backend/scripts/seed_demo.py (the tenant, its users, and the published policy) and docs/08-hackathon/demo-assets/contracts/ (the two contract PDFs).